# 1. What Is Flow Control?

Flow control determines which statements execute and how many times they execute inside a stored program. Conditional statements make decisions; loops repeat statements.

| Category | Purpose | Statements |
|---|---|---|
| Decision-making | Choose statements based on a condition. | IF, IF ... ELSE, ELSEIF, CASE |
| Repetition | Execute statements repeatedly. | LOOP, WHILE, REPEAT |
| Loop control | Exit or skip iterations. | LEAVE, ITERATE |

Real-life example: If a student scores at least 40, mark the result as Pass; otherwise, mark it as Fail. That is decision-making. If a calculation must be performed for each of 100 students, repetition may be useful.

# 2. IF ... THEN

Use IF ... THEN when statements should execute only if a condition is true.

```
IF condition THEN
    statements;
END IF;
```

## Example: Check pass status

```
DELIMITER //

CREATE PROCEDURE check_pass(IN p_marks INT)
BEGIN
    IF p_marks >= 40 THEN
        SELECT 'Pass' AS result;
    END IF;
END //

DELIMITER ;

CALL check_pass(65);
CALL check_pass(25);
```

CALL check_pass(65) returns Pass. CALL check_pass(25) produces no result row because the condition is false and no ELSE block is present.

MySQL stored-program IF statements end with END IF;. This is different from the IF() function used in SQL expressions.

# 3. IF ... ELSE

Use ELSE when one action is needed if the condition is true and another if it is false.

```
IF condition THEN
    statements_if_true;
```

```
ELSE
    statements_if_false;
END IF;
```

### Example: Even or odd

```
DELIMITER //

CREATE PROCEDURE check_even_odd(IN p_num INT)
BEGIN
    IF MOD(p_num, 2) = 0 THEN
        SELECT 'Even' AS result;
    ELSE
        SELECT 'Odd' AS result;
    END IF;
END //

DELIMITER ;

CALL check_even_odd(10);
CALL check_even_odd(7);
```

MOD(a, b) returns the remainder when a is divided by b. The function performs the calculation; IF makes the decision.

## 4. IF … ELSEIF … ELSE

Use ELSEIF when several conditions must be checked in sequence. MySQL checks conditions from top to bottom and executes the first matching branch.

```
IF condition1 THEN
    statements1;
ELSEIF condition2 THEN
    statements2;
ELSEIF condition3 THEN
    statements3;
ELSE
    default_statements;
END IF;
```

### Example: Grade calculation

For this teaching example, use the following illustrative grading scheme. It is not intended to represent a particular institution's official rules.

```
delimiter  //



create procedure calculate_grade (In std_id int)
Begin

declare s_name varchar(30);
declare s_marks int ;
declare s_grade varchar(10);

-- get details of the student from database
select name, marks
into s_name, s_marks
from student where student_id=std_id;

-- calculate grade
If s_marks is null then
select 'marks are invalid' as message;

elseif s_marks >=90 then
set s_grade ='A';



elseif s_marks >=75 then
set s_grade ='B';

elseif s_marks >= 60 then
set s_grade ='C';

elseif s_marks >=40 then
set s_grade ='D';

else
```

```
set s_grade='F';


end if;


if s_marks is not null then
select
 std_id as student_id,
 s_name as student_name,
 s_marks as marks,
 s_grade as grade;
 end if;
end //
delimiter ;
```

```
mysql> create procedure calculate_grade (In std_id int)
    -> Begin
    ->
    -> declare s_name varchar(30);
    -> declare s_marks int ;
    -> declare s_grade varchar(10);
    ->
    -> -- get details of the student from database
    -> select name, marks
    -> into s_name, s_marks
    -> from student where student_id=std_id;
    ->
    -> -- calculate grade
    -> If s_marks is null then
    -> select 'marks are invalid' as message;
    ->
    -> elseif s_marks >=90 then
    -> set s_grade ='A';
    ->
    ->
    -> elseif s_marks >=75 then
    -> set s_grade ='B';
    ->
    -> elseif s_marks >= 60 then
    -> set s_grade ='C';
    ->
    -> elseif s_marks >=40 then
    -> set s_grade ='D';
    ->
    -> else
    ->
    -> set s_grade='F';
    ->
    -> end if;
    ->
    -> if s_marks is not null then
    -> select
    ->  std_id as student_id,
    ->  s_name as student_name,
    ->  s_marks as marks,
    ->  s_grade as grade;
    ->  end if;
    ->
    ->
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> call calculate_grade(101);
+------------+--------------+-------+-------+
| student_id | student_name | marks | grade |
+------------+--------------+-------+-------+
|        101 | Amit         |    86 | B     |
+------------+--------------+-------+-------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> call calculate_grade(102);
+------------+--------------+-------+-------+
| student_id | student_name | marks | grade |
+------------+--------------+-------+-------+
|        102 | Neha         |    92 | A     |
+------------+--------------+-------+-------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> insert into student ( 106, 'Akash' , 10, 30 );
ERROR 1064 (42000): You have an error in your SQL syntax; check the m
mysql> insert into student  values ( 106, 'Akash' , 10, 30 );
Query OK, 1 row affected (0.01 sec)

mysql> select * from student;
+------------+-------+---------+-------+
| student_id | name  | dept_id | marks |
+------------+-------+---------+-------+
|        101 | Amit  |      10 | 85.50 |
|        102 | Neha  |      20 | 92.00 |
|        103 | Rahul |      10 | 76.50 |
|        104 | Priya |      30 | 88.00 |
|        106 | Akash |      10 | 30.00 |
+------------+-------+---------+-------+
5 rows in set (0.00 sec)

mysql> call calculate_grade(106);
+------------+--------------+-------+-------+
| student_id | student_name | marks | grade |
+------------+--------------+-------+-------+
|        106 | Akash        |    30 | F     |
+------------+--------------+-------+-------+
```

# 5. CASE Statement

CASE is another way to select one branch from several alternatives. MySQL stored programs support simple CASE and searched CASE.

## 5.1 Simple CASE

Simple CASE compares one expression with a list of values.

```
CASE p_dept_id
    WHEN 10 THEN
        SELECT 'Computer Science' AS department_name;
    WHEN 20 THEN
        SELECT 'Information Technology' AS department_name;
    WHEN 30 THEN
        SELECT 'Electronics' AS department_name;
    ELSE
```

```
        SELECT 'Unknown department' AS department_name;
END CASE;
```

### 5.2 Searched CASE

Searched CASE evaluates separate conditions. It is useful when the branches use comparisons or ranges.

```
CASE
    WHEN p_marks >= 90 THEN
        SELECT 'A' AS grade;
    WHEN p_marks >= 80 THEN
        SELECT 'B' AS grade;
    WHEN p_marks >= 70 THEN
        SELECT 'C' AS grade;
    ELSE
        SELECT 'F' AS grade;
END CASE;
```

This is the same general decision as the IF ... ELSEIF example, expressed using CASE. In a stored-program CASE statement, include ELSE or ensure a condition always matches; otherwise, no matching branch can raise a CASE-not-found error.

| Simple CASE | Searched CASE |
|---|---|
| Compares one expression with values. | Checks independent Boolean conditions. |
| WHEN 10, WHEN 20, etc. | WHEN p_marks >= 90, etc. |
| Good for matching fixed values. | Good for ranges and complex conditions. |

Do not confuse the stored-program CASE statement (which ends with END CASE) with the CASE expression used inside SELECT statements, which ends with END.

## 6. Why Do We Need Loops?

A loop repeats a set of statements. For example, to calculate 1 + 2 + 3 + 4 + 5, a loop can add each number to a running total.

Every counting loop normally needs three things:

1. Initialization: set the starting value, such as i = 1.
2. Condition or exit rule: decide when the loop should stop.
3. Update: change the counter, such as i = i + 1.

If the counter never changes and the exit condition never becomes true, the procedure may run indefinitely. Always identify initialization, condition and update before writing a loop.

## 7. LOOP with LEAVE

LOOP repeats its body until a LEAVE statement exits the loop. Unlike WHILE and REPEAT, LOOP does not have a built-in condition in its opening line, so you normally write an IF condition and LEAVE inside it.

```
[label_name:] LOOP
    statements;
```

```
    IF exit_condition THEN
        LEAVE label_name;
    END IF;
END LOOP [label_name];
```

## Example: Print numbers from 1 to 5

```
delimiter //

create procedure print_numbers_lnloop()

Begin

declare i int default 1;

number_loop: Loop

select i as number;

set i=i +1;


if i>5 then

leave number_loop;


end if;


end loop number_loop;


END //

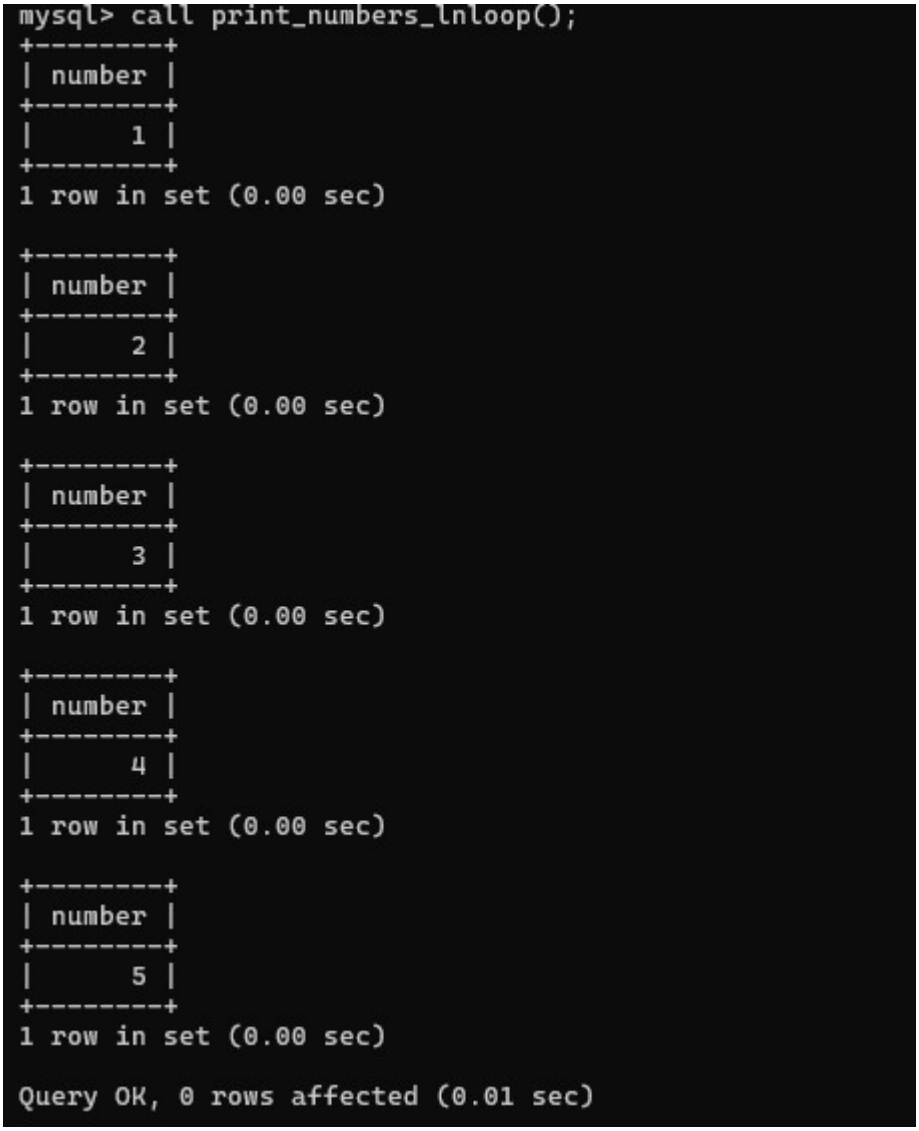

delimiter ;
```

The loop displays 1, 2, 3, 4 and 5. After displaying 5, i becomes 6; the condition i > 5 is true and LEAVE exits the loop.

The label number_loop identifies the loop. A label is particularly useful when a procedure contains nested loops.

```
mysql> delimiter //
mysql>
mysql> create procedure print_numbers_lnloop()
    -> Begin
    ->
    -> declare i int default 1;
    ->
    -> number_loop: Loop
    -> select i as number;
    -> set i=i +1;
    ->
    -> if i>5 then
    -> leave number_loop;
    ->
    -> end if;
    ->
    -> end loop number_loop;
    ->
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
```




# 8. WHILE ... DO

WHILE checks its condition before executing the loop body. If the condition is false at the beginning, the body executes zero times.

```
WHILE condition DO
    statements;
END WHILE;
```

## Example: Print numbers from 1 to 5

```
delimiter //


create procedure print_numbers_while()

begin


declare i int default 1;
```

```
while i<=5 do
select i as numbers;

set i=i+1;

end while;

end //

delimiter ;
```

The condition i <= 5 is checked before each iteration. After i becomes 6, the condition is false and the loop ends.

Remember to update i inside the loop. If i remains 1, the condition remains true and the loop will not terminate.

```
mysql> create procedure print_numbers_while()
    -> begin
    ->
    -> declare i int default 1;
    ->
    -> while i<=5 do
    -> select i as numbers;
    ->
    -> set i=i+1;
    ->
    -> end while;
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> call print_numbers_while();
+---------+
| numbers |
+---------+
|       1 |
+---------+
1 row in set (0.00 sec)

+---------+
| numbers |
+---------+
|       2 |
+---------+
1 row in set (0.00 sec)

+---------+
| numbers |
+---------+
|       3 |
+---------+
1 row in set (0.00 sec)

+---------+
| numbers |
+---------+
|       4 |
```

## 9. REPEAT … UNTIL

REPEAT executes its body first and checks the UNTIL condition at the end. It repeats until that condition becomes true. Therefore, its body executes at least once.

```
REPEAT
    statements;
UNTIL exit_condition
END REPEAT;
```

## Example: Print numbers from 1 to 5

```
delimiter //

create procedure print_numbers_repeat1()

begin

declare i int default 1;

repeat

select i as num;


set i=i+1;


until i>5

end repeat;


end //


delimiter ;
```

After printing 5, i becomes 6. The UNTIL condition i > 5 is true, so the loop ends. Unlike IF and WHILE conditions, the REPEAT statement does not put a semicolon after UNTIL's condition; END REPEAT terminates the construct.

If the exit condition is already true before the first iteration, REPEAT still executes its body once before checking the condition.

```
mysql> delimiter //
mysql>
mysql> create procedure print_numbers_repeat()
    -> begin
    ->
    -> declare i int default 1;
    ->
    -> repeat
    -> select i as num;
    ->
    -> set i=i+1;
    ->
    -> until i>5
    -> end repeat;
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> delimiter ;
mysql>
mysql> call print_numbers_repeat();
+------+
| num  |
+------+
|    1 |
+------+
1 row in set (0.00 sec)

+------+
| num  |
+------+
|    2 |
+------+
1 row in set (0.00 sec)

+------+
| num  |
+------+
|    3 |
+------+
1 row in set (0.00 sec)
```

# 10. WHILE vs REPEAT: Entry-Controlled and Exit-Controlled

| Feature | WHILE | REPEAT |
|---|---|---|
| When condition is checked | Before the body | After the body |
| Control type | Entry-controlled | Exit-controlled |
| Minimum executions | Zero | One |
| Continues while / until | Continues while condition is true | Continues until exit condition becomes true |
| Useful when | The body may not need to run at all | The body must run at least once |

Teaching analogy: WHILE is like checking whether a classroom is open before entering. REPEAT is like performing an action once and then checking whether it is time to stop.

Do not write UNTIL i <= 5 when intending to print 1 through 5 with the example above. The loop stops when the UNTIL condition becomes true, not while it is true.

# 11. LEAVE and ITERATE with Labels

## LEAVE

LEAVE exits the labeled loop or labeled compound block. When used with a loop label, it ends that loop immediately.

## ITERATE

ITERATE skips the remaining statements in the current iteration and starts the next iteration of the named loop. It is similar to continue in languages such as C, Java and JavaScript.

```
label_name: WHILE condition DO
    statements;

    IF skip_condition THEN
        ITERATE label_name;
    END IF;

    more_statements;
END WHILE label_name;
```

## Example: Print numbers 1 to 10, skipping 5

```
delimiter //

create procedure print_numbers_skip5()

begin

declare i int  default 1;

number_loop: while i < 10 do  - - 1< 10

set i = i+1;

if i=5  then

iterate number_loop;


end if;


select i as num;


end while number_loop;


end //
```

```
delimiter ;
```

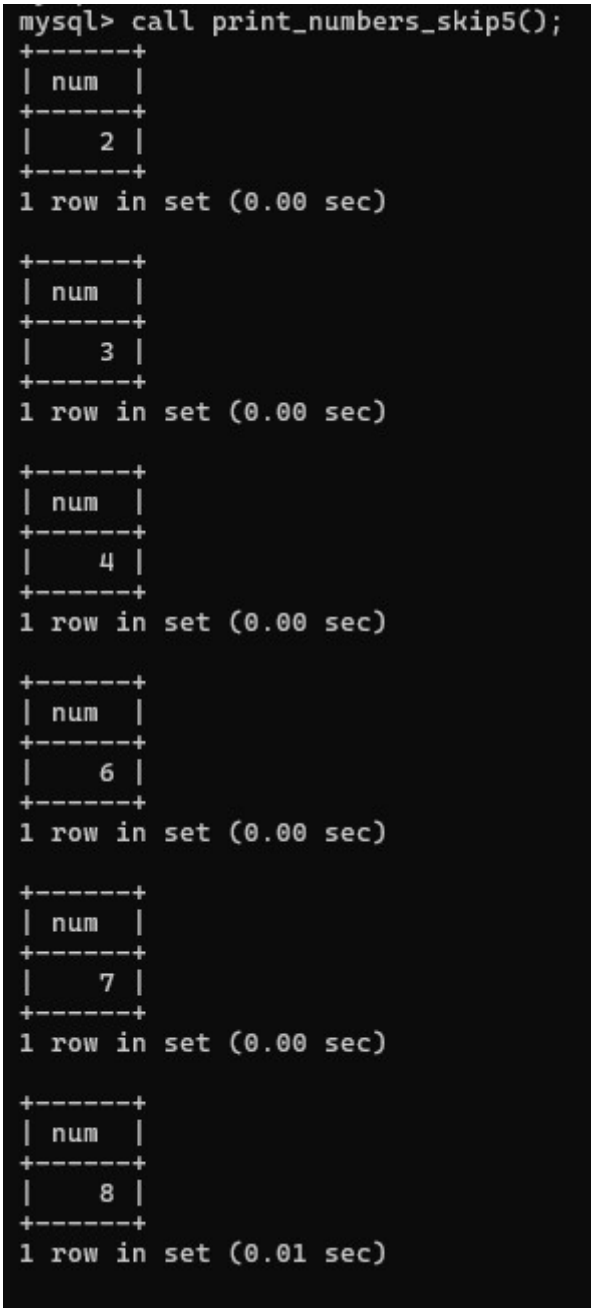
Output values: 1, 2, 3, 4, 6, 7, 8, 9, 10. When i equals 5, ITERATE skips the SELECT and moves to the next iteration.

Notice that i is incremented before ITERATE. If the counter update were placed after ITERATE, the value could remain 5 and the loop could repeat forever.

| Statement | Effect |
|---|---|
| LEAVE label_name | Exit the named loop or labeled block. |
| ITERATE label_name | Skip the rest of the current loop iteration and begin the next one. |

```
mysql> delimiter //
mysql>
mysql> create procedure print_numbers_skip5()
    ->
    -> begin
    ->
    -> declare i int default 1;
    ->
    -> number_loop: while i < 10 do
    ->
    -> set i = i+1;
    ->
    -> if i=5 then
    -> iterate number_loop;
    ->
    -> end if;
    ->
    -> select i as num;
    ->
    -> end while number_loop;
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> delimiter ;
```




# 12. Nested Loops and Nested Blocks

A nested loop is a loop inside another loop. The inner loop runs for each iteration of the outer loop unless control flow exits it.

## Example: Print a 2 × 3 coordinate pattern

```
delimiter //

create procedure nested_loop_demo()

begin

 declare i int default 1;

 declare j int;
```

```
outer_loop: while i <= 2 do
set j=1;

inner_loop: while j <= 3 do
select i as rows_number, j as column_number;

set j=j+1;
end while inner_loop;

set i=i+1;

end while outer_loop;

end //

delimiter ;
```

The inner counter j is reset to 1 for every new outer iteration. The procedure produces the coordinate pairs (1,1), (1,2), (1,3), (2,1), (2,2), and (2,3).

A labeled LEAVE can exit a specific outer loop from inside a nested loop. For example, LEAVE outer_loop; exits the outer loop, while LEAVE inner_loop; exits only the inner loop.

```
mysql> delimiter //
mysql>
mysql> create procedure nested_loop_demo()
    -> begin
    ->  declare i int default 1;
    ->  declare j int;
    ->
    ->  outer_loop: while i <= 2 do
    ->  set j=1;
    ->
    ->  inner_loop: while j <= 3 do
    ->  select i as rows_number, j as column_numbe
    ->
    ->  set j=j+1;
    ->  end while inner_loop;
    ->
    ->  set i=i+1;
    ->
    ->  end while outer_loop;
    ->
    ->  end //
Query OK, 0 rows affected (0.01 sec)

mysql>  delimiter ;
```

```
mysql> call nested_loop_demo();
+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           1 |             1 |
+-------------+---------------+
1 row in set (0.00 sec)

+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           1 |             2 |
+-------------+---------------+
1 row in set (0.00 sec)

+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           1 |             3 |
+-------------+---------------+
1 row in set (0.00 sec)

+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           2 |             1 |
+-------------+---------------+
1 row in set (0.00 sec)

+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           2 |             2 |
+-------------+---------------+
1 row in set (0.01 sec)

+-------------+---------------+
| rows_number | column_number |
+-------------+---------------+
|           2 |             3 |
+-------------+---------------+
1 row in set (0.01 sec)
```

## Nested blocks

BEGIN ... END can also be nested. Each block can have its own local variables, declared at the start of that block. A variable declared in an inner block is local to that block.

```
DELIMITER //

CREATE PROCEDURE nested_block_demo()
BEGIN
    DECLARE v_outer INT DEFAULT 10;

    BEGIN
        DECLARE v_inner INT DEFAULT 20;

        SELECT v_outer AS outer_value,
               v_inner AS inner_value;
    END;
```

```
    SELECT v_outer AS outer_value;
END //

DELIMITER ;
```

The inner block can access v_outer because it is declared in the enclosing block. The outer block cannot access v_inner after the inner block ends. Nested blocks are useful for organizing code and limiting variable scope.

# . Practice Questions

4. Write a procedure that checks whether a number is positive, negative or zero.
5. Write a procedure that returns the larger of two numbers.
6. Use a searched CASE statement to assign grades to marks.
7. Use a simple CASE statement to display a department name for department IDs 10, 20 and 30.
8. Use LOOP and LEAVE to display numbers from 1 to 10.
9. Use WHILE to calculate the sum of even numbers from 2 through 20.
10. Use REPEAT to display numbers from 1 to 5.
11. Display numbers from 1 to 20, skipping multiples of 3 by using ITERATE.
12. Use nested loops to display a 3 × 3 coordinate pattern.
13. Insert 20 test rows using a stored procedure and a loop.
14. Explain the difference between WHILE and REPEAT.
15. Explain the difference between LEAVE and ITERATE.
16. Why should a loop counter be initialized and updated?
17. What happens if the UNTIL condition in REPEAT is already true before the first iteration?

# . Final Revision Sheet

| Statement | Main purpose | Key point |
|---|---|---|
| IF ... THEN | Run statements when a condition is true. | No ELSE branch required. |
| IF ... ELSE | Choose between two branches. | One branch executes. |
| ELSEIF | Check additional conditions. | Order conditions carefully. |
| CASE | Select a branch from alternatives. | Simple CASE matches values; searched CASE tests conditions. |
| LOOP | Repeat statements. | Usually needs LEAVE to terminate. |
| WHILE | Repeat while condition is true. | Entry-controlled; may execute zero times. |
| REPEAT | Repeat until exit condition is true. | Exit-controlled; executes at least once. |
| LEAVE | Exit a labeled loop or block. | Ends the selected loop/block. |
| ITERATE | Skip the rest of this iteration. | Starts the next iteration of the selected loop. |

# What is a Function?

A function is a built-in operation that accepts input, performs processing, and returns a value. Easy idea: Input → Function → Result.

```
SELECT UPPER('hello');
SELECT ROUND(45.678, 2);
```

## 1. LOWER() and UPPER()

LOWER() converts text to lowercase. UPPER() converts text to uppercase.

```
SELECT LOWER('HELLO WORLD');
SELECT UPPER('hello world');
SELECT UPPER(student_name) FROM student;
```

## 2. SUBSTRING()

Extracts part of a string. MySQL string positions start from 1.

```
SELECT SUBSTRING('DATABASE', 1, 4);
-- DATA

SELECT SUBSTRING('DATABASE', 5, 4);
-- BASE
```

Syntax: SUBSTRING(string, start_position, length).

## 3. LEFT() and RIGHT()

LEFT() returns characters from the left. RIGHT() returns characters from the right.

```
SELECT LEFT('DATABASE', 4);
-- DATA

SELECT RIGHT('DATABASE', 4);
-- BASE
```

## 4. TRIM()

Removes leading and trailing spaces. It does not remove spaces between words.

```
SELECT TRIM('   Amit Sharma   ');
-- Amit Sharma
```

## 5. REPLACE()

Replaces one piece of text with another.

```
SELECT REPLACE('I like Java', 'Java', 'MySQL');
-- I like MySQL

SELECT REPLACE(email, '@gmail.com', '@company.com')
FROM student;
```

## 6. INSTR()

Finds the position of a substring. If it is not found, the result is 0.

```
SELECT INSTR('amit@gmail.com', '@');
-- 5
```

```
SELECT INSTR('Hello', 'x');
-- 0
```

### 7. LPAD() and RPAD()

LPAD() adds characters on the left until the requested total length is reached. RPAD() adds characters on the right.

```
SELECT LPAD('123', 5, '0');
-- 00123

SELECT RPAD('123', 5, '0');
-- 12300

SELECT LPAD(student_id, 5, '0') FROM student;
```

### 8. REVERSE()

Reverses the characters in a string.

```
SELECT REVERSE('MYSQL');
-- LQSYM

SELECT student_name, REVERSE(student_name)
FROM student;
```

### 9. String Functions — Quick Summary

| Function | Purpose |
|---|---|
| LOWER() | Convert to lowercase |
| UPPER() | Convert to uppercase |
| SUBSTRING() | Extract part of a string |
| LEFT() | Get characters from left |
| RIGHT() | Get characters from right |
| TRIM() | Remove leading/trailing spaces |
| REPLACE() | Replace text |
| INSTR() | Find position of text |
| LPAD() | Add padding on left |
| RPAD() | Add padding on right |
| REVERSE() | Reverse text |

## PART 2 — NUMERIC FUNCTIONS

Numeric functions perform calculations or transformations on numbers.

### 10. ROUND()

Rounds a number to the specified number of decimal places.

```
SELECT ROUND(45.6789, 2);
-- 45.68

SELECT ROUND(45.6789, 0);
-- 46
```

## 11. TRUNCATE()

Removes decimal digits without rounding.

```
SELECT TRUNCATE(45.6789, 2);
-- 45.67
```

Key difference: ROUND(45.6789, 2) → 45.68; TRUNCATE(45.6789, 2) → 45.67.

## 12. CEIL() and FLOOR()

CEIL() returns the smallest integer greater than or equal to the value. FLOOR() returns the largest integer less than or equal to the value.

```
SELECT CEIL(45.2);  -- 46
SELECT FLOOR(45.8); -- 45
```

## 13. MOD()

Returns the remainder after division.

```
SELECT MOD(10, 3); -- 1
SELECT MOD(10, 2); -- 0
```

Useful for even/odd checks: MOD(number, 2) = 0 means even.

## 14. ABS(), POWER(), SQRT()

```
SELECT ABS(-25);   -- 25
SELECT POWER(2, 3); -- 8
SELECT SQRT(25); -- 5
```

## 15. RAND()

Generates a random floating-point value between 0 and 1. It can produce a different value on different calls, so it is not deterministic.

```
SELECT RAND();
```

## 16. Numeric Functions — Quick Summary

| Function | Purpose |
|---|---|
| ROUND() | Round a number |
| TRUNCATE() | Remove decimal digits without rounding |
| CEIL() | Round upward |
| FLOOR() | Round downward |
| MOD() | Return remainder |
| ABS() | Absolute value |
| POWER() | Raise number to a power |
| SQRT() | Square root |
| RAND() | Random value |

# PART 3 — DATE AND TIME FUNCTIONS

DATE stores a date, TIME stores a time, and DATETIME stores both.

```
DATE     → 2026-10-14
TIME     → 14:30:25
DATETIME → 2026-10-14 14:30:25
```

## 17. NOW(), CURDATE(), CURTIME()

```
SELECT NOW();      -- current date + time
SELECT CURDATE();  -- current date
SELECT CURTIME();  -- current time
```

| Function | Returns |
|---|---|
| NOW() | Current date and time |
| CURDATE() | Current date |
| CURTIME() | Current time |

## 18. YEAR(), MONTH(), DAY()

```
SELECT YEAR('2026-10-14');  -- 2026
SELECT MONTH('2026-10-14'); -- 10
SELECT DAY('2026-10-14');   -- 14
```

## 19. DAYNAME() and LAST_DAY()

```
SELECT DAYNAME('2026-10-14'); -- Wednesday
SELECT LAST_DAY('2026-02-10'); -- 2026-02-28
```

LAST_DAY() returns the last date of the month.

## 20. DATE_FORMAT()

Formats a date into a desired display format.

```
SELECT DATE_FORMAT('2026-10-14', '%d-%m-%Y');
-- 14-10-2026

SELECT DATE_FORMAT('2026-10-14', '%M %d, %Y');
-- October 14, 2026
```

| Code | Meaning |
|---|---|
| %Y | 4-digit year |
| %m | Month number |
| %d | Day |
| %M | Month name |
| %H | Hour |
| %i | Minute |
| %s | Second |

## 21. STR_TO_DATE()

Converts a string into a date according to the supplied format.

```
SELECT STR_TO_DATE('14-10-2026', '%d-%m-%Y');
-- 2026-10-14
```

DATE_FORMAT() is used to format a date for output; STR_TO_DATE() parses a string into a date.

## 22. DATEDIFF()

Returns the difference between two dates in days.

```
SELECT DATEDIFF('2026-10-14', '2026-10-01');
-- 13
```

DATEDIFF(date1, date2) calculates date1 − date2 in days.

## 23. DATE_ADD() and DATE_SUB()

DATE_ADD() adds an interval. DATE_SUB() subtracts an interval.

```
SELECT DATE_ADD('2026-10-14', INTERVAL 10 DAY);
-- 2026-10-24

SELECT DATE_SUB('2026-10-14', INTERVAL 10 DAY);
-- 2026-10-04

SELECT DATE_ADD('2026-10-14', INTERVAL 2 MONTH);
SELECT DATE_ADD('2026-10-14', INTERVAL 1 YEAR);
```

## 24. TIMESTAMPDIFF()

Calculates the difference between two date/time values in a specified unit.

```
SELECT TIMESTAMPDIFF(
    YEAR,
    '2000-05-10',
    '2026-10-14'
);
-- 26
```

The unit can be YEAR, MONTH, DAY, HOUR and other supported units. DATEDIFF() gives days; TIMESTAMPDIFF() lets you choose the unit.

## 25. Date Functions — Quick Summary

| Function | Purpose |
|---|---|
| NOW() | Current date + time |
| CURDATE() | Current date |
| CURTIME() | Current time |
| DATE_FORMAT() | Format date |
| STR_TO_DATE() | String → date |
| DATEDIFF() | Difference in days |
| DATE_ADD() | Add interval |
| DATE_SUB() | Subtract interval |
| TIMESTAMPDIFF() | Difference in selected unit |
| YEAR() | Extract year |
| MONTH() | Extract month |
| DAY() | Extract day |
| DAYNAME() | Day name |
| LAST_DAY() | Last date of month |

```
mysql> select student_id, Upper(name) from student;
+------------+-------------+
| student_id | Upper(name) |
+------------+-------------+
|        101 | AMIT PATIL  |
|        102 | NEHA SHINDE |
|        103 | RAHUL KHOT  |
|        104 | PRIYA BAPAT |
|        106 | AKASH       |
+------------+-------------+
5 rows in set (0.00 sec)

mysql> select student_id, lower(name) from student;
+------------+-------------+
| student_id | lower(name) |
+------------+-------------+
|        101 | amit patil  |
|        102 | neha shinde |
|        103 | rahul khot  |
|        104 | priya bapat |
|        106 | akash       |
+------------+-------------+
5 rows in set (0.00 sec)

mysql> update student set name = 'Akash yadav' where student_id=106;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> select substring('DATABASE', 1, 4);
+----------------------------+
| substring('DATABASE', 1, 4) |
+----------------------------+
| DATA                       |
+----------------------------+
1 row in set (0.00 sec)

mysql> select substring ('DATABASE', 5, 4);
ERROR 1630 (42000): FUNCTION oct14.substring does
mysql> select substring('DATABASE', 5, 4);
+----------------------------+
| substring('DATABASE', 5, 4) |
+----------------------------+
| BASE                       |
+----------------------------+
1 row in set (0.00 sec)

mysql> select substring('DATABASE', -5, 4);
+-----------------------------+
| substring('DATABASE', -5, 4) |
+-----------------------------+
| ABAS                        |
+-----------------------------+
1 row in set (0.00 sec)

mysql> select left('DATABASE',4);
+--------------------+
| left('DATABASE',4) |
+--------------------+
| DATA               |
+--------------------+
1 row in set (0.00 sec)

mysql>  select right('DATABASE', 4);
```

```
mysql> update student set name ='   Amit Patil  ' where student_id=
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from student;
+------------+-----------------+---------+-------+
| student_id | name            | dept_id | marks |
+------------+-----------------+---------+-------+
|        101 |    Amit Patil   |      10 | 85.50 |
|        102 | Neha shinde     |      20 | 92.00 |
|        103 | Rahul khot      |      10 | 76.50 |
|        104 | Priya bapat     |      30 | 88.00 |
|        106 | Akash yadav     |      10 | 30.00 |
+------------+-----------------+---------+-------+
5 rows in set (0.00 sec)

mysql> select trim('    Amit patil   ');
+--------------------------+
| trim('    Amit patil   ') |
+--------------------------+
| Amit patil               |
+--------------------------+
1 row in set (0.00 sec)

mysql> select trim(name) from student where student_id=101;
+------------+
| trim(name) |
+------------+
| Amit Patil |
+------------+
1 row in set (0.00 sec)

mysql> select replace('I like java', 'java' , 'mysql');
+-----------------------------------------+
| replace('I like java', 'java' , 'mysql') |
+-----------------------------------------+
| I like mysql                            |
+-----------------------------------------+
1 row in set (0.00 sec)

mysql> select replace('I like java and java', 'java' , 'mysql');
```

```
mysql> select replace(name, 'shinde', 'chaudhari') from student;
+-------------------------------------+
| replace(name, 'shinde', 'chaudhari') |
+-------------------------------------+
|    Amit Patil                       |
| Neha chaudhari                      |
| Rahul khot                          |
| Priya bapat                         |
| Akash yadav                         |
+-------------------------------------+
5 rows in set (0.00 sec)

mysql> select * from student;
+------------+-----------------+---------+-------+
| student_id | name            | dept_id | marks |
+------------+-----------------+---------+-------+
|        101 |    Amit Patil   |      10 | 85.50 |
|        102 | Neha shinde     |      20 | 92.00 |
|        103 | Rahul khot      |      10 | 76.50 |
|        104 | Priya bapat     |      30 | 88.00 |
|        106 | Akash yadav     |      10 | 30.00 |
+------------+-----------------+---------+-------+
5 rows in set (0.00 sec)
```

```
mysql> select instr('aniket@gmail.com', '@') as position;
+----------+
| position |
+----------+
|        7 |
+----------+
1 row in set (0.00 sec)

mysql> select instr('aniket@gmail.com', 'y') as position;
+----------+
| position |
+----------+
|        0 |
+----------+
1 row in set (0.00 sec)

mysql> select instr('aniket@gmail.com', 'a') as position;
+----------+
| position |
+----------+
|        1 |
+----------+
1 row in set (0.00 sec)

mysql> select student_id , name, instr(name, 't') as tposition  from student;
+------------+-----------------+-----------+
| student_id | name            | tposition |
+------------+-----------------+-----------+
|        101 |    Amit Patil   |         7 |
|        102 | Neha chaudhari  |         0 |
|        103 | Rahul khot      |        10 |
|        104 | Priya bapat     |        11 |
|        106 | Akash yadav     |         0 |
+------------+-----------------+-----------+
5 rows in set (0.00 sec)

mysql> select student_id , name, instr(name, 'patil') as tposition  from student;
+------------+-----------------+-----------+
| student_id | name            | tposition |
+------------+-----------------+-----------+
|        101 |    Amit Patil   |         9 |
|        102 | Neha chaudhari  |         0 |
|        103 | Rahul khot      |         0 |
|        104 | Priya bapat     |         0 |
```

```
mysql> select Lpad('123', 5, '0');
+---------------------+
| Lpad('123', 5, '0') |
+---------------------+
| 00123               |
+---------------------+
1 row in set (0.00 sec)

mysql> select Reverse('MYSQL');
+------------------+
| Reverse('MYSQL') |
+------------------+
| LQSYM            |
+------------------+
1 row in set (0.00 sec)

mysql> select round(20.867, 2);
+------------------+
| round(20.867, 2) |
+------------------+
|            20.87 |
+------------------+
1 row in set (0.00 sec)

mysql> select truncate (53.34589 , 3);
+-------------------------+
| truncate (53.34589 , 3) |
+-------------------------+
|                  53.345 |
+-------------------------+
1 row in set (0.00 sec)

mysql> select round(20.8641, 2);
+-------------------+
| round(20.8641, 2) |
+-------------------+
|             20.86 |
+-------------------+
1 row in set (0.00 sec)

mysql> select ceil(45);
```

```
mysql> select ceil(45);
+----------+
| ceil(45) |
+----------+
|       45 |
+----------+
1 row in set (0.00 sec)

mysql> select floor(45);
+-----------+
| floor(45) |
+-----------+
|        45 |
+-----------+
1 row in set (0.00 sec)

mysql> select Mod(14, 3);
+------------+
| Mod(14, 3) |
+------------+
|          2 |
+------------+
1 row in set (0.00 sec)

mysql> select sqrt(25);
+----------+
| sqrt(25) |
+----------+
|        5 |
+----------+
1 row in set (0.00 sec)

mysql> select sqrt(30);
+-------------------+
| sqrt(30)          |
+-------------------+
| 5.477225575051661 |
+-------------------+
1 row in set (0.00 sec)
```

```
mysql> select sqrt(-25);
+-----------+
| sqrt(-25) |
+-----------+
|      NULL |
+-----------+
1 row in set (0.00 sec)

mysql> select rand();
+--------------------+
| rand()             |
+--------------------+
| 0.9063102266809998 |
+--------------------+
1 row in set (0.00 sec)

mysql> select rand();
+---------------------+
| rand()              |
+---------------------+
| 0.30131013812619273 |
+---------------------+
1 row in set (0.00 sec)

mysql> select rand();
+--------------------+
| rand()             |
+--------------------+
| 0.7876200413216092 |
+--------------------+
1 row in set (0.00 sec)

mysql> select rand();
+--------------------+
| rand()             |
+--------------------+
| 0.0341698229631128 |
+--------------------+
1 row in set (0.00 sec)
```

```
mysql> select NOW();
+---------------------+
| NOW()               |
+---------------------+
| 2026-10-15 13:30:39 |
+---------------------+
1 row in set (0.00 sec)

mysql> select curdate()
    -> ^C
mysql> select curdate();
+------------+
| curdate()  |
+------------+
| 2026-10-15 |
+------------+
1 row in set (0.00 sec)

mysql> select curtime();
+-----------+
| curtime() |
+-----------+
| 13:31:14  |
+-----------+
1 row in set (0.00 sec)

mysql> select year (now());
+--------------+
| year (now()) |
+--------------+
|         2026 |
+--------------+
1 row in set (0.00 sec)

mysql> select month(now());
+--------------+
| month(now()) |
+--------------+
|           10 |
+--------------+
1 row in set (0.00 sec)
```

```
mysql> select date_format('2026-10-15', '%d-%m-%y');
+--------------------------------------+
| date_format('2026-10-15', '%d-%m-%y') |
+--------------------------------------+
| 15-10-26                             |
+--------------------------------------+
1 row in set (0.00 sec)

mysql> select date_format('2026-10-15', '%m %d, %y');
+--------------------------------------+
| date_format('2026-10-15', '%m %d, %y') |
+--------------------------------------+
| 10 15, 26                            |
+--------------------------------------+
1 row in set (0.00 sec)

mysql> select date_format('2026-10-15', '%M %d, %Y');
+--------------------------------------+
| date_format('2026-10-15', '%M %d, %Y') |
+--------------------------------------+
| October 15, 2026                     |
+--------------------------------------+
1 row in set (0.00 sec)

mysql> select str_to_date('15-10-2026', '%d-%m-%y');
+--------------------------------------+
| str_to_date('15-10-2026', '%d-%m-%y') |
+--------------------------------------+
| 2020-10-15                           |
+--------------------------------------+
1 row in set, 1 warning (0.00 sec)

mysql> select str_to_date('15-10-2026', '%d-%m-%y');
+--------------------------------------+
| str_to_date('15-10-2026', '%d-%m-%y') |
+--------------------------------------+
| 2020-10-15                           |
+--------------------------------------+
1 row in set, 1 warning (0.00 sec)

mysql> select str_to_date('15-10-2026', '%d-%m-%Y');
+--------------------------------------+
| str_to_date('15-10-2026', '%d-%m-%Y') |
```

```
mysql> select str_to_date('15-10-2026', '%d-%m-%Y');
+--------------------------------------+
| str_to_date('15-10-2026', '%d-%m-%Y') |
+--------------------------------------+
| 2026-10-15                           |
+--------------------------------------+
1 row in set (0.00 sec)

mysql> select datediff('2026-09-10' , '2026-10-10');
+---------------------------------------+
| datediff('2026-09-10' , '2026-10-10') |
+---------------------------------------+
|                                   -30 |
+---------------------------------------+
1 row in set (0.00 sec)

mysql> select datediff('2026-10-10' , '2026-09-10');
+---------------------------------------+
| datediff('2026-10-10' , '2026-09-10') |
+---------------------------------------+
|                                    30 |
+---------------------------------------+
1 row in set (0.00 sec)

mysql> select dateadd('2026-10-15', interval 10 day);
ERROR 1064 (42000): You have an error in your SQL syntax;
1
mysql> select date_add('2026-10-15', INTERVAL 10 DAY);
+----------------------------------------+
| date_add('2026-10-15', INTERVAL 10 DAY) |
+----------------------------------------+
| 2026-10-25                             |
+----------------------------------------+
1 row in set (0.00 sec)

mysql> select date_sub('2026-10-15', INTERVAL 10 DAY);
+----------------------------------------+
| date_sub('2026-10-15', INTERVAL 10 DAY) |
+----------------------------------------+
| 2026-10-05                             |
+----------------------------------------+
```